

EXPERIMENT NO: 01

EXPERIMENT NAME: Write a Python program to convert a given text to lowercase and remove all punctuation.

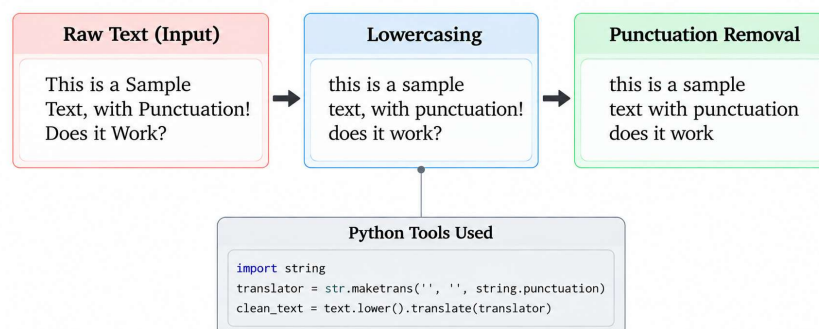
OBJECTIVES

- To understand the concept of text normalization in Natural Language Processing (NLP).
- To convert all uppercase characters in a text to lowercase using Python's lower() method.
- To remove punctuation marks from text efficiently using string.punctuation and translate() method.

THEORY

Text preprocessing is an essential first step in Natural Language Processing (NLP). Raw text data often contains uppercase letters and punctuation marks that increase vocabulary size and create redundant tokens.

- **Lowercasing:** Converting text to lowercase ensures case-insensitivity so that words like 'Text', 'TEXT', and 'text' are treated as identical tokens.
- **Punctuation Removal:** Removing punctuation marks (such as !, ?, ,, " -) eliminates unnecessary special characters that do not contribute to core word semantics.
- **string.punctuation & translate():** Python's string.punctuation provides a pre-defined string of all ASCII punctuation characters. Combined with str.maketrans() and translate(), it allows fast character removal.



ALGORITHM

Step 1: Import Python's standard string module.

Step 2: Take text input from the user using input().

Step 3: Convert the entire text to lowercase using text.lower().

Step 4: Create a translation table to map all punctuation characters to None using str.maketrans('', '', string.punctuation).

Step 5: Apply the translation table using text.translate() to obtain the cleaned text.

Step 6: Display the resulting cleaned text to the console.

SOURCE CODE

```
import string

text = input("Enter a text: ")

# Convert to Lowercase
text = text.lower()

# Remove punctuation
text = text.translate(str.maketrans('', '', string.punctuation))

print("Result:", text)
```

INPUT & OUTPUT

Sample 1

Input:

Enter a text: Hi !! This is Taz, Currently pursuing B.Sc. in Engineering in ICE

Output:

Result: hi this is taz currently pursuing bsc in engineering in ice

Sample 2

Input:

Enter a text: Python is easy to learn! It is widely used in Data Science.

Output:

Result: python is easy to learn it is widely used in data science

EXPERIMENT NO: 02

EXPERIMENT NAME: Write a Python program to tokenize a paragraph into sentences and words using NLTK.

OBJECTIVES

- To understand the concept and significance of tokenization in Natural Language Processing (NLP).
- To implement sentence tokenization and word tokenization using the NLTK library.
- To learn how tokenization breaks down unstructured text corpora into basic textual units for analysis.

THEORY

Tokenization is one of the fundamental preprocessing steps in Natural Language Processing (NLP). It involves breaking down a large body of unstructured text (such as paragraphs) into smaller, manageable textual units called tokens.

- **Sentence Tokenization (sent_tokenize):** Splits a continuous paragraph into individual sentence units based on sentence boundary markers like periods, question marks, and exclamation points.
- **Word Tokenization (word_tokenize):** Breaks down text into discrete word tokens, numbers, and punctuation symbols, forming the structural units for vocabulary building.
- **NLTK Punkt Tokenizer:** Uses pre-trained unsupervised models to accurately identify sentence and word boundaries even in complex punctuation scenarios.

ALGORITHM

Step 1: Import the nltk module and tokenize functions (sent_tokenize, word_tokenize) from nltk.tokenize.

Step 2: Download the required NLTK tokenizer resources ('punkt' and 'punkt_tab').

Step 3: Prompt the user to enter an input paragraph.

Step 4: Apply sent_tokenize(paragraph) to segment the paragraph into sentences.

Step 5: Display the segmented sentences line by line.

Step 6: Apply word_tokenize(paragraph) to extract individual word tokens.

Step 7: Display the resulting list of word tokens.

SOURCE CODE

```
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize

# Download required NLTK resources
nltk.download('punkt')
nltk.download('punkt_tab')

# Input paragraph
paragraph = input("Enter a paragraph: ")

# Sentence tokenization
sentences = sent_tokenize(paragraph)

print("\nSentences:")
for sentence in sentences:
    print(sentence)

# Word tokenization
words = word_tokenize(paragraph)

print("\nWords:")
print(words)
```

INPUT & OUTPUT

Sample 1

Enter a paragraph: Natural Language Processing is an important field of Artificial Intelligence. It helps computers understand human language. I am learning NLP in my university.

Sentences:

Natural Language Processing is an important field of Artificial Intelligence. It helps computers understand human language. I am learning NLP in my university.

Words:

['Natural', 'Language', 'Processing', 'is', 'an', 'important', 'field', 'of', 'Artificial', 'Intelligence', '.', 'It', 'helps', 'computers', 'understand', 'human', 'language', '.', 'I', 'am', 'learning', 'NLP', 'in', 'my', 'university', '.']

EXPERIMENT NO: 03

EXPERIMENT NAME: Write a Python program to remove English stopwords from tokenized words using NLTK.

OBJECTIVES

- To understand the role and impact of stop words in Natural Language Processing (NLP).
- To implement stop word removal using NLTK's stopwords corpus and word_tokenize.
- To reduce feature space and eliminate high-frequency, low-information functional words from text corpora.

THEORY

Stop words are extremely common grammatical words in a natural language (such as 'is', 'the', 'at', 'which', and 'and') that occur with high frequency but carry negligible domain-specific semantic meaning for analytical tasks.

- **Stop Word Removal:** Filtering out stop words eliminates structural noise, reduces feature dimensionality, and improves processing speed for NLP tasks like Text Classification and Search Indexing.
- **NLTK Stopwords Corpus:** Provides pre-compiled standard lists of common stop words across multiple languages, accessible via `stopwords.words('english')`.
- **Case-Insensitive Filtering:** Converting tokens to lowercase during stop word comparison ensures accurate matching regardless of sentence capitalization.

ALGORITHM

Step 1: Import nltk, stopwords from nltk.corpus, and word_tokenize from nltk.tokenize.

Step 2: Download the required NLTK resources ('punkt', 'punkt_tab', and 'stopwords').

Step 3: Prompt the user to enter an input sentence.

Step 4: Tokenize the input sentence into individual word tokens using `word_tokenize(text)`.

Step 5: Load the standard set of English stop words using `set(stopwords.words('english'))`.

Step 6: Iterate through the token list and retain only those tokens whose lowercased form is not in the stop word set.

Step 7: Display the original tokenized words list and the final filtered words list.

SOURCE CODE

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Download required resources
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')

# Take input from user
text = input("Enter a sentence: ")

# Tokenize the sentence
words = word_tokenize(text)

# Get English stopwords
stop_words = set(stopwords.words('english'))

# Remove stopwords
filtered_words = []

for word in words:
    if word.lower() not in stop_words:
        filtered_words.append(word)

# Display output
print("\nTokenized Words:")
print(words)

print("\nAfter Removing Stopwords:")
print(filtered_words)
```

INPUT & OUTPUT

Sample 1

Enter a sentence: I am learning NLP because it is very useful for understanding human language.

Tokenized Words: ['I', 'am', 'learning', 'NLP', 'because', 'it', 'is', 'very', 'useful', 'for', 'understanding', 'human', 'language', '.']

After Removing Stopwords: ['learning', 'NLP', 'useful', 'understanding', 'human', 'language', '.']

EXPERIMENT NO: 04

EXPERIMENT NAME: Write a Python program to apply stemming to words using NLTK's PorterStemmer.

OBJECTIVES

- To understand the concept and goal of stemming in Natural Language Processing (NLP).
- To implement word stemming using NLTK's PorterStemmer class.
- To reduce inflected and derived word forms to their common base or root stems.

THEORY

Stemming is a text normalization technique in Natural Language Processing (NLP) that reduces words to their base or root form by stripping common morphological suffixes (such as '-ing', '-ed', '-es', '-ly').

- **Porter Stemmer Algorithm:** Developed by Martin Porter, it is a rule-based algorithmic stemmer that applies five sequential phases of word suffix removal rules.
- **Morphological Normalization:** Maps grammatical variations of a word (e.g., 'programming', 'programmer', 'programs') to a single stem ('program').
- **NLP Performance Impact:** Reduces overall vocabulary size and increases feature matching recall in Search Engines, Text Mining, and Information Retrieval.

ALGORITHM

Step 1: Import nltk, PorterStemmer from nltk.stem, and word_tokenize from nltk.tokenize.

Step 2: Download the required NLTK tokenizer resources ('punkt' and 'punkt_tab').

Step 3: Instantiate the PorterStemmer object (stemmer = PorterStemmer()).

Step 4: Prompt the user to enter an input sentence.

Step 5: Tokenize the sentence into individual word tokens using word_tokenize(text).

Step 6: Iterate through the token list and apply stemmer.stem(word) to reduce each token to its root stem.

Step 7: Display the original word list and the resulting list of stemmed words.

SOURCE CODE

```
import nltk
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

# Download required resource
nltk.download('punkt')
nltk.download('punkt_tab')

# Create Porter Stemmer object
stemmer = PorterStemmer()

# Take input from user
text = input("Enter a sentence: ")

# Tokenize the sentence
words = word_tokenize(text)

# Apply stemming
stemmed_words = []

for word in words:
    stemmed_words.append(stemmer.stem(word))

# Display output
print("\nOriginal Words:")
print(words)

print("\nStemmed Words:")
print(stemmed_words)
```

INPUT & OUTPUT

Input: Enter a sentence: The programmer is programming and programs rapidly in Python.

Output:

Original Words:

```
['The', 'programmer', 'is', 'programming', 'and', 'programs', 'rapidly', 'in', 'Python', '.']
```

```
Stemmed Words:['the', 'program', 'is', 'program', 'and', 'program', 'rapidli', 'in', 'python', '.']
```

EXPERIMENT NO: 05

EXPERIMENT NAME: Write a Python program to apply lemmatization to words using NLTK's WordNetLemmatizer.

OBJECTIVES

- To understand the concept and advantages of lemmatization over stemming in Natural Language Processing (NLP).
- To implement verb lemmatization using NLTK's WordNetLemmatizer with the pos='v' parameter.
- To reduce inflected verb forms to their base dictionary lemmas (canonical forms) using morphological analysis.

THEORY

Lemmatization is a text normalization technique in Natural Language Processing (NLP) that reduces inflected word forms to their dictionary base form, known as a lemma, using vocabulary knowledge and morphological analysis.

- **Lemmatization vs. Stemming:** While stemming heuristically cuts off word suffixes (often resulting in non-words), lemmatization accurately returns meaningful, valid dictionary root words.
- **WordNet Lexical Database:** NLTK's WordNetLemmatizer maps words to their canonical lemmas by referencing the large WordNet English lexical database.
- **Verb Part-of-Speech Tagging (pos='v'):** Passing pos='v' explicitly instructs the lemmatizer to treat tokens as verbs, transforming inflected verb forms like 'running' -> 'run', 'writing' -> 'write', and 'ate' -> 'eat'.

ALGORITHM

Step 1: Import nltk, WordNetLemmatizer from nltk.stem, and word_tokenize from nltk.tokenize.

Step 2: Download the required NLTK resources ('punkt', 'punkt_tab', and 'wordnet').

Step 3: Instantiate the WordNetLemmatizer object (lemmatizer = WordNetLemmatizer()).

Step 4: Prompt the user to enter an input sentence.

Step 5: Tokenize the sentence into individual word tokens using word_tokenize(text).

Step 6: Iterate through the token list and apply `lemmatizer.lemmatize(word, pos='v')` to convert each verb token to its root lemma.

Step 7: Display the original token list and the resulting list of lemmatized words.

SOURCE CODE

```
import nltk
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

# Download required resources
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('wordnet')

# Create Lemmatizer
lemmatizer = WordNetLemmatizer()

# Take input from user
text = input("Enter a sentence: ")

# Tokenize
words = word_tokenize(text)

# Lemmatization
lemmatized_words = []

for word in words:
    lemma = lemmatizer.lemmatize(word, pos='v')
    lemmatized_words.append(lemma)

# Output
print("\nOriginal Words:")
print(words)

print("\nLemmatized Words:")
print(lemmatized_words)
```

INPUT & OUTPUT

Sample 1

Enter a sentence: I am learning Natural Language Processing and it helps me understand human language.

Original Words:

['I', 'am', 'learning', 'Natural', 'Language', 'Processing', 'and', 'it', 'helps', 'me', 'understand', 'human', 'language', '.']

Lemmatized Words:

['I', 'be', 'learn', 'Natural', 'Language', 'Processing', 'and', 'it', 'help', 'me', 'understand', 'human', 'language', '.']

Sample 2

Enter a sentence: Students are studying and researchers are developing new NLP applications.

Original Words:

['Students', 'are', 'studying', 'and', 'researchers', 'are', 'developing', 'new', 'NLP', 'applications', '.']

Lemmatized Words:

['Students', 'be', 'study', 'and', 'researchers', 'be', 'develop', 'new', 'NLP', 'applications', '.']

EXPERIMENT NO: 06

EXPERIMENT NAME: Write a Python program to perform Part-of-Speech (POS) tagging on a sentence using NLTK.

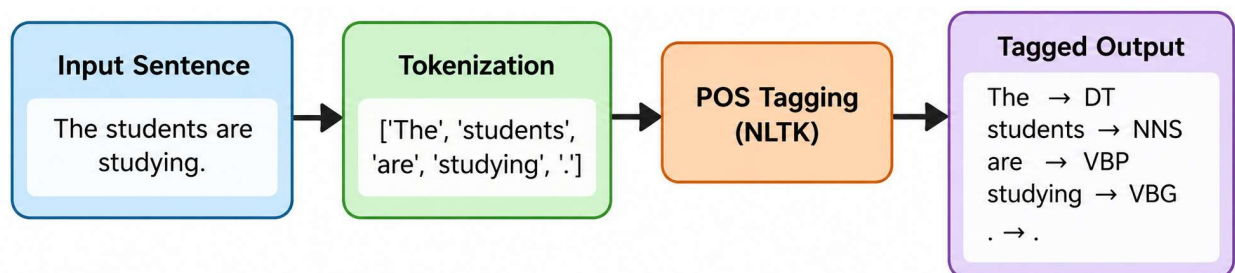
OBJECTIVES

- To understand the concept and significance of Part-of-Speech (POS) tagging in Natural Language Processing (NLP).
- To implement POS tagging using NLTK's `nltk.pos_tag()` function and the Averaged Perceptron Tagger.
- To analyze syntactic roles of words (such as nouns, verbs, adjectives, adverbs) in natural language sentences.

THEORY

Part-of-Speech (POS) tagging is the process of categorizing words in a text corpus into their corresponding grammatical classes (such as Nouns, Verbs, Adjectives, Adverbs, and Prepositions) based on word definition and sentence context.

- **Syntactic Context & Disambiguation:** POS tagging uses sentence structure to resolve lexical ambiguity, distinguishing when a word functions as a Noun (e.g., 'a book') versus a Verb (e.g., 'to book a flight').
- **Penn Treebank Tagset:** NLTK utilizes the standard Penn Treebank tagset notation (e.g., NN = Noun, VB = Verb, JJ = Adjective, RB = Adverb, DT = Determiner).
- **Downstream NLP Applications:** POS tags provide critical syntactic features for Named Entity Recognition (NER), Dependency Parsing, Information Extraction, and Text-to-Speech synthesis.



ALGORITHM

Step 1: Import `nltk` and `word_tokenize` from `nltk.tokenize`.

Step 2: Download the required NLTK resources ('punkt', 'punkt_tab', 'averaged_perceptron_tagger', and 'averaged_perceptron_tagger_eng').

Step 3: Prompt the user to enter an input sentence.

Step 4: Tokenize the input sentence into individual word tokens using `word_tokenize(text)`.

Step 5: Apply `nltk.pos_tag(words)` to assign grammatical POS tags to each token based on context.

Step 6: Display the resulting list of (word, POS_tag) tuples to the console.

SOURCE CODE

```
import nltk
from nltk.tokenize import word_tokenize

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')

# Take input from user
text = input("Enter a sentence: ")

# Tokenize
words = word_tokenize(text)

# POS tagging
pos_tags = nltk.pos_tag(words)

# Output
print(pos_tags)
```

INPUT & OUTPUT

Sample 1

Enter a sentence: Natural Language Processing is an important field of Artificial Intelligence.

Output: [('Natural', 'JJ'), ('Language', 'NNP'), ('Processing', 'NNP'), ('is', 'VBZ'), ('an', 'DT'), ('important', 'JJ'), ('field', 'NN'), ('of', 'IN'), ('Artificial', 'NNP'), ('Intelligence', 'NNP'), (',', ','), (',', ',')]

EXPERIMENT NO: 07

EXPERIMENT NAME: Write a Python program to show the most frequent words in a text using NLTK FreqDist.

OBJECTIVES

- To understand the concept and applications of Word Frequency Distribution in Natural Language Processing (NLP).
- To implement word frequency analysis using NLTK's FreqDist class from nltk.probability.
- To identify and extract the top most frequent word tokens from a given text corpus using .most_common().

THEORY

Frequency Distribution analysis is a foundational Statistical NLP technique used to count and analyze the occurrences of distinct word tokens across a body of text.

- **Term Frequency Analysis:** Counting word frequencies helps identify prominent keywords, main topics, and vocabulary distribution patterns within textual datasets.
- **NLTK FreqDist Class:** NLTK provides the FreqDist class in nltk.probability, which functions as a specialized hash map to compute frequencies of tokenized inputs efficiently.
- **Zipf's Law & Keyword Extraction:** Frequency distributions form the basis of Zipf's Law evaluation and TF-IDF (Term Frequency - Inverse Document Frequency) calculations in Information Retrieval.

ALGORITHM

Step 1: Import nltk, word_tokenize from nltk.tokenize, and FreqDist from nltk.probability.

Step 2: Download the required NLTK resources ('punkt' and 'punkt_tab').

Step 3: Prompt the user to enter an input text.

Step 4: Tokenize the input text into word tokens using word_tokenize(text).

Step 5: Compute word frequencies by instantiating fdist = FreqDist(words).

Step 6: Extract the top 5 most frequent word tokens using fdist.most_common(5).

Step 7: Display the resulting list of (word, frequency) tuples to the console.

SOURCE CODE

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.probability import FreqDist

# Download required resources
nltk.download('punkt')
nltk.download('punkt_tab')

# Take input from user
text = input("Enter a text: ")

# Tokenize the text
words = word_tokenize(text)

# Calculate word frequency
fdist = FreqDist(words)

# Display 5 most frequent words
print("\nMost Frequent Words:")
print(fdist.most_common(5))
```

INPUT & OUTPUT

Sample 1

Enter a text: Natural Language Processing is an important field. NLP is useful for many applications.

Output:

Most Frequent Words:

[('NLP', 2), ('.', 2), ('Natural', 1), ('Language', 1), ('Processing', 1)]

Sample 2

Enter a text: Python is easy to learn. Python is powerful and Python is popular.

Output:

Most Frequent Words: [('Python', 3), ('is', 2), ('.', 2), ('easy', 1), ('to', 1)]

EXPERIMENT NO: 08

EXPERIMENT NAME: Write a Python program to generate bigrams (2-word sequences) from a text using NLTK.

OBJECTIVES

- To understand the concept and utility of N-grams (specifically Bigrams) in Natural Language Processing (NLP).
- To implement Bigram generation using NLTK's `bigrams()` utility from `nltk.util`.
- To analyze contiguous 2-word sequences for language modeling and contextual word association.

THEORY

In Natural Language Processing (NLP), an N-gram is a contiguous sequence of n items (words or tokens) extracted from a given text corpus. A Bigram represents a 2-gram, which is a pair of consecutive adjacent words.

- **N-gram Contextual Modeling:** Unlike unigrams (single words), bigrams capture short-range contextual relationships and word co-occurrences in human language.
- **NLTK `bigrams()` Function:** NLTK provides `bigrams()` in `nltk.util`, which generates an iterator over sliding windows of 2 consecutive tokens across an input list.
- **Applications in NLP:** Bigram models form the foundation of Statistical Language Modeling, Auto-completion / Next-word prediction, Machine Translation, and Collocation Detection.

ALGORITHM

Step 1: Import `nltk`, `word_tokenize` from `nltk.tokenize`, and `bigrams` from `nltk.util`.

Step 2: Download the required NLTK resources ('punkt' and 'punkt_tab').

Step 3: Prompt the user to enter an input text.

Step 4: Tokenize the input text into word tokens using `word_tokenize(text)`.

Step 5: Generate bigrams by applying `bigrams(words)` and converting the result into a list.

Step 6: Display the resulting list of 2-word tuple pairs to the console.

SOURCE CODE

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.util import bigrams

nltk.download('punkt')
nltk.download('punkt_tab')

# Take input from user
text = input("Enter a text: ")

# Tokenize the text
words = word_tokenize(text)

# Generate bigrams
bigram_list = list(bigrams(words))

# Display the bigrams
print("\nBigrams:")
print(bigram_list)
```

INPUT & OUTPUT

Sample 1

Enter a text: Natural Language Processing is an important field.

Output:

Bigrams:

```
[('Natural', 'Language'), ('Language', 'Processing'), ('Processing', 'is'), ('is', 'an'), ('an', 'important'), ('important', 'field'), (',', '.)]
```

Sample 2

Enter a text: I am learning NLP in my university.

Bigrams:

```
[('I', 'am'), ('am', 'learning'), ('learning', 'NLP'), ('NLP', 'in'), ('in', 'my'), ('my', 'university'), ('university', '.')]
```

EXPERIMENT NO: 09

EXPERIMENT NAME: Write a Python program to perform sentiment analysis (Positive/Negative/Neutral) using NLTK's VADER.

OBJECTIVES

- To understand the concepts and mechanisms of Sentiment Analysis in Natural Language Processing (NLP).
- To implement rule-based sentiment classification using NLTK's VADER (Valence Aware Dictionary and sEntiment Reasoner).
- To analyze polarity scores (pos, neg, neu, compound) to classify text into Positive, Negative, or Neutral categories.

THEORY

Sentiment Analysis is an NLP technique used to extract subjective opinion, emotional tone, and polarity (positive, negative, or neutral) expressed within written textual data.

- **VADER Lexicon-Based Analyzer:** VADER (Valence Aware Dictionary and sEntiment Reasoner) is a rule-based sentiment analysis model specifically tuned for social media posts, product reviews, and general English prose.
- **Polarity Score Breakdown:** VADER computes four metrics: pos (positive proportion), neg (negative proportion), neu (neutral proportion), and compound (normalized aggregated sentiment score from -1.0 to +1.0).
- **Classification Thresholds:** A compound score ≥ 0.05 indicates Positive sentiment, ≤ -0.05 indicates Negative sentiment, and values between -0.05 and 0.05 represent Neutral sentiment.

ALGORITHM

Step 1: Import nltk and SentimentIntensityAnalyzer from nltk.sentiment.

Step 2: Download the required NLTK resource ('vader_lexicon').

Step 3: Instantiate the SentimentIntensityAnalyzer object (sia = SentimentIntensityAnalyzer()).

Step 4: Prompt the user to enter an input text.

Step 5: Calculate sentiment polarity scores using sia.polarity_scores(text).

Step 6: Extract the compound score and evaluate sentiment conditions (Positive if ≥ 0.05 , Negative if ≤ -0.05 , else Neutral).

Step 7: Display the polarity scores dictionary and the final sentiment classification.

SOURCE CODE

```
import nltk
from nltk.sentiment import SentimentIntensityAnalyzer

nltk.download('vader_lexicon', quiet=True)

# Create VADER analyzer
sia = SentimentIntensityAnalyzer()

# Take input from user
text = input("Enter a text: ")

# Calculate sentiment scores
scores = sia.polarity_scores(text)

compound = scores['compound']
if compound >= 0.05:
    sentiment = "Positive"

elif compound <= -0.05:
    sentiment = "Negative"

else:
    sentiment = "Neutral"

# Display results
print("\nScores:", scores)
print("Overall Sentiment:", sentiment)
```

INPUT & OUTPUT

Enter a text: I love this NLP course. It is amazing and very useful!

Output:

Scores: {'neg': 0.0, 'neu': 0.541, 'pos': 0.459, 'compound': 0.8932}

Overall Sentiment: Positive

EXPERIMENT NO: 10

EXPERIMENT NAME: Write a Python program to perform basic Named Entity Recognition (NER) and chunking using NLTK.

OBJECTIVES

- To understand the concepts of Named Entity Recognition (NER) and Information Chunking in Natural Language Processing (NLP).
- To implement NER using NLTK's `ne_chunk()` function combined with POS tagging.
- To extract and classify real-world named entities (such as Person, Location, Organization, GPE) from unstructured text into syntax trees.

THEORY

Named Entity Recognition (NER) is a fundamental Information Extraction task in Natural Language Processing (NLP) that identifies and categorizes key entities (such as names of people, places, organizations, and geographical entities) mentioned in unstructured text.

- **Chunking (Shallow Parsing):** Chunking groups individual words and POS tags into higher-level syntactic chunks (such as Noun Phrases or Entity Chunks) without building a full syntactic parse tree.
- **Named Entity Categories:** NLTK classifies named entities into predefined categories including PERSON (e.g., 'Akbar'), GPE (Geopolitical Entity like 'India' or 'Agra'), and ORGANIZATION.
- **NLTK `ne_chunk()` Function:** Applies a trained Maximum Entropy classifier over POS-tagged tokens to build an NLTK Tree structure containing nested entity labels.

ALGORITHM

Step 1: Import `nltk`, `word_tokenize` from `nltk.tokenize`, and `pos_tag`, `ne_chunk` from `nltk`.

Step 2: Download the required NLTK resources ('`maxent_ne_chunker`', '`maxent_ne_chunker_tab`', and '`words`').

Step 3: Prompt the user to enter an input sentence.

Step 4: Tokenize the input sentence into word tokens using `word_tokenize(text)`.

Step 5: Perform Part-of-Speech tagging on the token list using `pos_tag(words)`.

Step 6: Pass the POS-tagged tokens to `ne_chunk(pos_tags)` to generate the Named Entity Chunk Tree.

Step 7: Display the resulting NER tree structure to the console.

SOURCE CODE

```
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag, ne_chunk

# Download NER resources
nltk.download('maxent_ne_chunker', quiet=True)
nltk.download('maxent_ne_chunker_tab', quiet=True)
nltk.download('words', quiet=True)

# Take input from user
text = input("Enter a sentence: ")

# Tokenize the sentence
words = word_tokenize(text)

# Perform POS tagging
pos_tags = pos_tag(words)

# Perform Named Entity Recognition and chunking
ner_tree = ne_chunk(pos_tags)

# Display the NER result
print(ner_tree)
```

INPUT & OUTPUT

Sample 1

Enter a sentence: Akbar was a great Mughal emperor. His son Jahangir became Emperor. Akbar ruled India and lived in Agra.

Output:

(S

(PERSON Akbar/NNP)

was/VBD

a/DT

great/JJ
Mughal/NNP
emperor/NN
./.
His/PRP\$
son/NN
(PERSON Jahangir/NNP)
became/VBD
Emperor/NNP
./.
(PERSON Akbar/NNP)
ruled/VBD
(GPE India/NNP)
and/CC
lived/VBD
in/IN
(GPE Agra/NNP)
./.)

Entity List

1. Akbar → (PERSON – Person)
2. Jahangir → (PERSON – Person)
3. Akbar → (PERSON – Person)
4. India → (GPE – Geopolitical Entity)
5. Agra → (GPE – Geopolitical Entity)